

PAM Role-Based Access Control (RBAC) Documentation

Overview

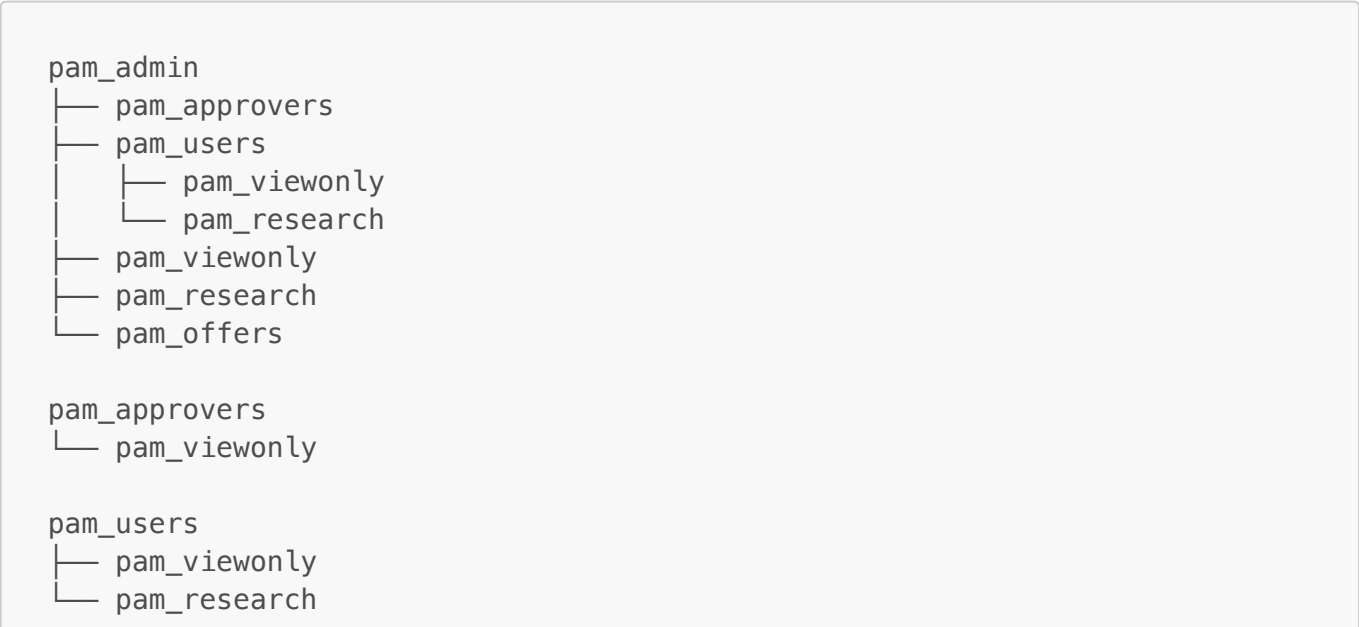
PAM implements Role-Based Access Control (RBAC) using Azure AD groups integrated with Azure App Service Easy Auth. This document describes the role hierarchy, route protection, and configuration.

Role Definitions

Role	Description	Target Users
pam_admin	Full administrative control, system settings, user management	System administrators
pam_users	Full edit access to promotions, promo code generation, research tools	Promo owners, analysts
pam_approvers	View-only + approval actions (device finance, revenue accounting)	Finance approvers
pam_viewonly	Read-only access to PAM homepage and reviewers tab	Auditors, reviewers
pam_research	Access to research workspace only	Research analysts
pam_offers	Access to offers workspace only	Offers operations team

Role Hierarchy

Roles inherit permissions from lower-privileged roles. A user with a higher role automatically has access to routes protected by inherited roles.



```
pam_viewonly (no inheritance)

pam_research (isolated - no inheritance)

pam_offers (isolated - no inheritance)
```

Hierarchy Matrix

User's Role	Can Access Routes Protected By
pam_admin	pam_admin, pam_approvers, pam_users, pam_viewonly, pam_research, pam_offers
pam_users	pam_users, pam_viewonly, pam_research
pam_approvers	pam_approvers, pam_viewonly
pam_viewonly	pam_viewonly only
pam_research	pam_research only
pam_offers	pam_offers only

Route Protection by Blueprint

Admin Blueprint (/admin/*)

Required Role: pam_admin

Route	Description
/admin/dashboard	Admin dashboard
/admin/users	User management
/admin/test-connections	Test external connections
/admin/azure-diagnostics	Azure connectivity diagnostics
/admin/sql-connection-reset	Reset SQL connection pool
/admin/cache-status	View cache status
/admin/cache-refresh	Force cache refresh
/admin/delete-promo	Delete promotions
/admin/date-diagnostics	Date validation diagnostics
/admin/data-health	Data health status
All other /admin/* routes	Admin-only

Research Blueprint (/research/*)

Required Role: pam_research

Route	Description
/research/	Research workspace home
/research/pete	PETE research tool
/research/pete/chat	PETE chat interface
/research/api/main-data	Main data API
/research/api/promo-error-reasons	Promo error lookup
/research/api/rate-plans	Rate plan data
/research/api/eip-by-ban	EIP lookup by BAN
All other /research/* routes	Research-only

Promo Blueprint (/promo/*)

Required Role: pam_users (most routes), pam_viewonly (reviewers)

Route	Required Role	Description
/promo/rdc	pam_users	RDC promotions page
/promo/spe	pam_users	SPE promotions page
/promo/rebates	pam_users	Rebates page
/promo/date-mismatch	pam_users	Date mismatch tool
/promo/edit-spe/<code>	pam_users	Edit SPE promotion
/promo/capacity	pam_users	Capacity planning
/promo/updates	pam_users	Updates page
/promo/generate-sql-*	pam_users	SQL generation tools
/promo/approvers	pam_viewonly	Reviewers/approvers tab

API Blueprint (/api/*)

Required Role: pam_users

Route	Description
/api/get_promo_details/<code>	Get promo details
/api/get_promo_details_full/<code>	Get full promo payload
/api/jira_summary/<code>	Generate JIRA summary
/api/search_orbit/<id>	Search ORBIT by ID

Route	Description
/api/update_testing_status	Update test status
/api/generate_next_promo_code	Generate new promo code
/api/create_from_orbit	Create promo from ORBIT
All other /api/* routes	Requires pam_users

Core Blueprint (/)

Mixed Protection

Route	Required Role	Description
/	None	Redirects to landing
/landing	None	Main landing page
/PAM_homepage	pam_viewonly	PAM workspace home
/offers	None*	Offers workspace
/health	None	Health check endpoint
/theme	None	Theme preference setter
/debug/*	None*	Debug endpoints

Consider adding role protection to /offers and /debug/ routes.

Azure AD Group Configuration

Environment Variables

Configure these in your .env file or Azure App Service Configuration:

```
# Azure AD Group Object IDs
ENTRA_GROUP_PAM_ADMIN=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
ENTRA_GROUP_PAM_USERS=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
ENTRA_GROUP_PAM_VIEWONLY=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
ENTRA_GROUP_PAM_RESEARCH=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
ENTRA_GROUP_PAM_OFFERS=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
ENTRA_GROUP_PAM_APPROVERS=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
```

How Group Mapping Works

- 1. User authenticates via Azure Easy Auth
- 2. X-MS-CLIENT-PRINCIPAL header contains encoded user info including group memberships
- 3. auth.py decodes the header and extracts group object IDs
- 4. Group IDs are matched against environment variables to determine user's role

5. `@role_required` decorator checks if user's role has access to the route
-

Development & Testing

Dev Mode Configuration

For local development without Azure Easy Auth:

```
# .env file
DEV_MODE=true
DEV_USER_ROLE=pam_admin # Options: pam_admin, pam_users, pam_viewonly,
                        pam_research, pam_offers, pam_approvers
DEV_USER_NAME=Dev User
```

Testing Different Roles

1. Set `DEV_USER_ROLE` to the role you want to test
2. Restart the Flask app
3. The app will simulate that role for all requests

```
# Test as viewonly user
DEV_USER_ROLE=pam_viewonly

# Test as research user (isolated)
DEV_USER_ROLE=pam_research

# Test as approver
DEV_USER_ROLE=pam_approvers
```

Debug Endpoints

- `/debug/me` - View current user's identity and roles
 - `/debug/env` - Check which environment variables are configured
-

Implementation Details

The `@role_required` Decorator

Located in `auth.py`, this decorator protects routes:

```
from auth import role_required

@app.route('/protected-route')
@role_required('pam_users')
```

```
def protected_route():
    return 'Only pam_users and above can see this'
```

Role Checking Logic

```
# auth.py
ROLE_HIERARCHY = {
    'pam_admin': ['pam_admin', 'pam_approvers', 'pam_users',
    'pam_viewonly', 'pam_research', 'pam_offers'],
    'pam_approvers': ['pam_approvers', 'pam_viewonly'],
    'pam_users': ['pam_users', 'pam_viewonly', 'pam_research'],
    'pam_viewonly': ['pam_viewonly'],
    'pam_research': ['pam_research'],
    'pam_offers': ['pam_offers'],
}

def has_role(user_role: str, required_role: str) -> bool:
    """Check if user_role grants access to required_role."""
    allowed_roles = ROLE_HIERARCHY.get(user_role, [])
    return required_role in allowed_roles
```

Access Denied Behavior

When a user attempts to access a route they don't have permission for:

- 1. **API Routes** (/api/*): Returns HTTP 403 with JSON error
- 2. **Page Routes**: Returns HTTP 403 with error page

Files Modified for RBAC

File	Changes
auth.py	Role hierarchy, role_required decorator, has_role function
admin/routes.py	Added @role_required('pam_admin') to all routes
research/routes.py	Added @role_required('pam_research') to all routes
promo/routes.py	Added @role_required('pam_users') / @role_required('pam_viewonly')
api/routes.py	Added @role_required('pam_users') to all routes
core/__init__.py	Added @role_required('pam_viewonly') to PAM homepage

Recommended Enhancements

- 1. Lock Debug Endpoints (Optional)

Add admin protection to debug endpoints:

```
@core_bp.route('/debug/me', endpoint='debug_user')
@role_required('pam_admin')
def debug_user():
    ...
```

2. Lock Offers Workspace

```
@core_bp.route('/offers', endpoint='offers_workspace')
@role_required('pam_offers')
def offers_workspace():
    ...
```

3. UI-Level Permission Controls

Hide sidebar tabs/tiles for routes users can't access. This improves UX by not showing links to forbidden pages.

Troubleshooting

User Gets 403 Forbidden

- 1. Check user's Azure AD group memberships in Azure Portal
- 2. Verify `ENTRA_GROUP_*` environment variables are set correctly
- 3. Use `/debug/me` endpoint to see what role the user is assigned
- 4. Ensure the route has the correct `@role_required` decorator

Dev Mode Not Working

- 1. Verify `DEV_MODE=true` is set (lowercase `true`)
- 2. Check `DEV_USER_ROLE` is a valid role name
- 3. Restart the Flask app after changing `.env`

Role Not Being Detected

- 1. Check Azure Easy Auth is configured in App Service
- 2. Verify the app registration includes group claims
- 3. Check `X-MS-CLIENT-PRINCIPAL` header is being sent

Version History

Date	Version	Changes
2026-02-10	1.0	Initial RBAC implementation

Date	Version	Changes
2026-02-10	1.1	Added DEV_USER_ROLE for testing
2026-02-12	1.2	Documentation finalized